Chapter 2
Services and Clients Using JAX-RPC

This chapter describes the procedure to develop, assemble, and deploy RPC-based Web services in Sun Java System Application Server environment; how to build clients that invoke such services.

This chapter contains the following sections:

- JAX-RPC Implementation
- Developing JAX-RPC Web Services
- Assembling and Deploying JAX-RPC Web Services
- Invoking JAX-RPC Web Services
- JAX-RPC Client Invoking an EJB
- Building Security into JAX-RPC Web Services
- JAX-RPC Tools
- Java Language Types Supported By JAX-RPC

JAX-RPC Implementation

Java™ API for XML-based RPC (JAX-RPC) is an API for building Web services and clients that use remote procedure calls (RPC) and XML. The RPC mechanism enables clients to execute procedures on other systems in a distributed environment. In JAX-RPC, a remote procedure call is represented by an XML-based protocol such as SOAP. The SOAP specification defines envelope structure, encoding rules, and a convention for representing remote procedure calls and responses. These calls and responses are transmitted as SOAP messages over HTTP. For more information on SOAP messages, see SOAP Messages.

JAX-RPC uses technologies HTTP, SOAP, and the WSDL defined by the World Wide Web Consortium (W3C), which makes it possible

for a JAX-RPC client to access a Web service that is not running on the Java platform and vice versa. Sun Java System Application Server implementation of the JAX-RPC API uses HTTP as the transport protocol. The implementation also provides necessary tools to generate stubs, ties, and other artifacts needed on the client-side and the server-side. See JAX-RPC Tools.

Implementation of JAX-RPC in Sun Java System Application Server provides the following benefits to the developers:

- Enables JAX-RPC clients to invoke Web services developed across heterogeneous platforms.
- Developers are not exposed to the complexity of the underlying runtime mechanisms such as, SOAP protocol level mechanisms, marshalling, and unmarshalling. A JAX-RPC runtime system or a library abstracts these runtime mechanisms for the Web services programming model. This simplifies Web service development.
- Provides support for WSDL to Java and Java to WSDL mapping as part of the development of Web service's endpoints and clients. (A Web service's endpoint is the address at which the Web service can be reached using a specific protocol or a data format, from where its methods can be invoked.)
- Supports the J2SE SDK classes, application classes that you have written, and JavaBean components. For more information, see Java Language Types Supported By JAX-RPC.
- Enables a Web service endpoint to be developed using the Servlet model. A Web service endpoint is deployed on the application server. These endpoints are described using a WSDL document.
- A JAX-RPC client can use stubs-based, dynamic proxy, or dynamic invocation interface (DII) programming models to invoke a heterogeneous Web service endpoint. SeeInvoking JAX-RPC Web Services.
- Provides `wscompile` and `wsdeploy` tools to help in the development of Web services and clients. See JAX-RPC Tools.

Developing JAX-RPC Web Services

JAX-RPC Web services are synchronous services which means that, every time a client invokes a JAX-RPC Web services operation, it always receives a SOAP response, even if the method that implements the operation returns void. For more information on the Web services operation, see Messaging Models Used in Web Services.

Web services deployed to Sun Java System Application Server can be accessed by any type of client such as an application client, any J2EE component performing the role of a client, any J2SE-based client, or a .net client.

The following steps describe the procedure to create JAX-RPC Web services using the Java interface and its implementation:

1. Define a class that represents the remote interface to the service; this is the service endpoint interface. This class contains the signature for the methods that a client may invoke on the service. The service endpoint interface extends the `java.rmi.Remote` interface and its methods must throw `java.rmi.RemoteException`. The following code illustrates the creation of a service endpoint interface.

```
package hello;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface HelloIF extends Remote{

  public String sayHello(String S) throws RemoteException;

}
```

In the code illustration above, the name of the package file is `hello`, and the service definition interface is `HelloIF.java`.

A service endpoint is deployed in a container that implements the JAX-RPC runtime system.

2. Write the service implementation class. The service implementation class is an ordinary Java class. Invocation is done inside the servelt container. The code illustration below shows how to write the service implementation class.

```
package hello;

public class HelloImpl implements HelloIF {

  public String message = "Hello";

  public String sayHello(String S) {

    return message + S;

  }
  }
```

3. In order to handle the communication between the client and the service endpoint, JAX-RPC needs various classes, interfaces, and other files on both the client-side and the server-side. JAX-RPC implementation in Sun Java System Application Server provides the `wscompile` tool to generate these artifacts.

   The `wscompile` tool uses the configuration file, `config.xml` to read the interface and implementation class, for generating client-side and server-side artifacts. The `wscompile` tool also creates the WSDL description for the service.

The configuration file of the example is given below:

```
<?xml version="1.0" encoding="UTF-8"?>

<configuration

xmlns="http://java.sun.com/xml/ns/jax-rpc/ri/config"> <service name=”HelloWorld”
    targetNameSpace=”http://hello.org/HelloWorld.wsdl”
    typeNameSpace=”http://hello.org/hello/type”

    packageName=”hello”>
    <interface name=”hello.HelloIF”
     servantName=”hello.HelloImpl”/>
  </service>

</configuration>
```

For information about the configuration file, see Configuration File.

For information about the XML schema for creating a configuration file, see Appendix A, "XML Schema Definitions."

The following is the syntax to run the `wscompile` tool:

```
wscompile -gen:both -d build/client -classpath build/shared config.xml
```

Stubs and ties are the most important artifacts that the `wscompile` tool generates. Stubs and ties are the classes that enable the communication between a service endpoint and a client. The stub class sits on the client side, between the service client and the JAX-RPC client runtime system. The stub class is responsible for converting a request from a JAX-RPC service client to a SOAP message and sending it across to the service endpoint using the specified protocol. It also converts the response from the service endpoint, which it receives in the form of a SOAP message, to the format required by the client. Converting a client request to SOAP format is called marshalling; converting back from SOAP format to a client response is unmarshalling.

Similarly, the tie class resides on the server side, between the service endpoint and the JAX-RPC runtime system. The tie class handles marshalling and unmarshalling the data between the service endpoint class and the SOAP format. A stub is a local object that acts as a proxy for the service endpoint.

You can use an `ant` build file (`build.xml`) to compile the service, generate server-side artifacts and create a portable war file. You can find a sample `build.xml` file at the following location:

install_dir/samples/webservices/jaxrpc/simple/src

For more information on creating a build.xml file, see Creating the build.xml File.

4. Assemble and deploy the service to Sun Java System Application Server. See Assembling and Deploying JAX-RPC Web Services.
5. Write the client side application that invokes the service. See Invoking JAX-RPC Web Services.

JAX-RPC Web Services Using a WSDL

You can create a JAX-RPC Web service using an existing WSDL document. In this method, the wscompile tool generates the service definition interface for the Web service using the WSDL. The WSDL portType is mapped to the Java service definition interface. To generate the service interface from the WSDL, use the wscompile command with -import option, passing it the location of the WSDL document. Alternatively, you can store the information required to generate the service definition interface in a configuration file by name config.xml. The config.xml, typically stores the location of the WSDL that you wish to access.

The following wscompile command reads the config.xml to generate the service definition interface:

```
wscompile -gen:server -import <config.xml>
```

The configuration file with a WSDL document has the following format:

```
<?xml version="1.0" encoding="UTF-8"?>

<configuration xmlns="http://java.sun.com/jax-rpc-ri/xrpcc-config">

<wsdl location="[1]"
  packageName="[2]">
  <typeMappingRegistry>[3] </typeMappingRegistry>

</wsdl>

</configuration>
```

The configuration file with a WSDL document has the following attributes:

- wsdl location - URL pointing to a WSDL document.
- packageName - Specifies a fully qualified name of the Java package for the generated classes/interfaces.

- `typeMappingRegistry` - The type mapping registry used for this service.

For information on the XML schema for creating a configuration file, see Appendix A, "XML Schema Definitions."

The code below is the configuration file of the sample and is located at: *install_dir*`/samples/webservices/jaxrpc/simple`

```
<?xml version="1.0" encoding="UTF-8"?>

<configuration xmlns="http://java.sun.com/xml/ns/jax-rpc/ri/config">

<wsdl location="HelloWorld.wsdl"

packageName="samples.webservices.jaxrpc.simple"/>

</configuration>
```

After you generate the service interface, perform Step 2 to Step 5 under the section, Developing JAX-RPC Web Services.

Assembling and Deploying JAX-RPC Web Services

A JAX-RPC Web service application can be assembled and deployed to Sun Java System Application Server as a WAR file. A WAR file contains the files needed for the web application in compressed form.

The following steps describe the procedure to assemble and deploy a Web services application to Sun Java System Application Server.

1. Create the WAR file. To create a WAR file that contains the service code, create a `build.xml` file, specifying the `create-war` command for the `target-name`. The following code is a sample `build.xml` file that creates a WAR file:

   ```
   <target name="create-war" depends="compile-server"

   description="Packages the WAR file">

   <echo message="Creating the WAR...."/>

   <delete file="../${portable-war}" />

   <delete dir="${assemble}/WEB-INF" />
   ```

```
<copy todir="${assemble}/WEB-INF/classes/">

<fileset dir="${build}/shared/" includes="**/*.class" />

</copy>

<copy file="web.xml" todir="${assemble}/WEB-INF" />

<copy file="jaxrpc-ri.xml" todir="${assemble}/WEB-INF" />

<jar jarfile="${assemble}/${portable-war}" >

<fileset dir="${assemble}" includes="WEB-INF/**" />

</jar>

<move file="${assemble}/${portable-war}" todir="../" />

</target>
```

This XML file when executed, bundles the files into a WAR file named `hello-portable.war`. This WAR file is not ready for deployment because it does not contain the tie classes. A WAR (web application archive) file contains a complete web application in compressed form.

A special directory under the document root, `WEB-INF`, contains everything related to the application that is not in the public document tree of the application. No file contained in `WEB-INF` can be served directly to the client. The contents of `WEB-INF` include:

- `/WEB-INF/classes/*`, the directory for servlet and other classes.
- `/WEB-INF/lib/*.jar`, the directory for JAR files containing beans and other utility classes.
- `/WEB-INF/web.xml` and `/WEB-INF/sun-web.xml`, XML-based deployment descriptors that specify the web application configuration, including mappings, initialization parameters, and security constraints.

  The web application directory structure follows the structure outlined in the J2EE specification.

  In the example, the `hello-portable.war` contains the following files:

- WEB-INF/classes/hello/`HelloIF.class`

    - WEB-INF/classes/hello/HelloImpl.class
    - WEB-INF/jaxrpc-ri.xml
    - WEB-INF/web.xml
2. Define the configuration file that specifies the name of the service and its service endpoint interface and the class. The name of the configuration file must be `jaxrpc-ri.xml`. The following configuration file is the configuration file of the example.

```
<?xml version="1.0" encoding="UTF-8"?>

<webServices

  xmlns="http://java.sun.com/xml/ns/jax-rpc/ri/dd"
  version="1.0"
  targetNamespaceBase="http://hello.org/wsdl"
  typeNamespaceBase="http://hello.org/types"
  urlPatternBase="/ws">

<endpoint

    name="HelloWorld"
    displayName="HelloWorld Service"
    description="A simple web service"
    interface="samples.webservices.jaxrpc.simple.HelloIF"
    implementation="samples.webservices.jaxrpc.simple. HelloImpl"/>

  <endpointMapping

      endpointName="HelloWorld"
      urlPattern="/simple"/>

</webServices>
```

The configuration file contains the following `webServices` attributes:

- The `webServices` element includes `name`, `typeNamespace`, and `targetNamespace` attributes.
    - The `name` attribute is used to generate the WSDL file for publication in a public registry.
    - The `typeNamespace` attribute defines the namespace in WSDL document for types generated by the `wscompile` tool.
    - The `targetNamespace` attribute is used for qualifying everything else in the WSDL document.

For information about the XML schema for creating a runtime configuration file, see Appendix A, "XML Schema Definitions."

3. Create `web.xml` deployment descriptor file to include the information required for deploying a service, such as mapping the service to an URL, specifying the location of the configuration file in the WAR file, etc. For more information on the deployment descriptors, see the *Sun Java System Application Server Developer's Guide*.

   For general information about DTD files and XML, see the XML specification at:

   http://www.w3.org/TR/REC-xml

   The following is the deployment descriptor of the example:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE web-app

PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"

"http://java.sun.com/dtd/web-app_2_3.dtd">

<web-app>

<display-name>Hello World Application</display-name>

<description>A web application containing a simple JAX-RPC endpoint</description>

<session-config>

<session-timeout>60</session-timeout>

</session-config>

</web-app>
```

   For information about the XML schema for creating deployment descriptors, see Appendix A, "XML Schema Definitions."

   Web services applications have a directory structure, all accessible from a mapping to the application's document root (for

example, /hello).

4. Use the wsdeploy tool to create a deployable WAR module. The wsdeploy tool executes the wscompile tool to generate the stubs, tie classes and other necessary classes.

   ```
   wsdeploy -keep tmpdir tempdir -o hello.war hello-portable.war
   ```

   The wsdeploy command when executed, performs the following tasks:

   - Reads the hello-portable.war file as input
   - Gets information from the jaxrpc-ri.xml file that's inside the hello-portable.war file
   - Generates the tie classes for the service
   - Generates a WSDL file named HelloWorld.wsdl
   - Assembles the tie classes, the HelloWorld.wsdl file, and the contents of hello-portable.war file into a deployable WAR file.

     See wsdeploy Tool for information on using the wsdeploy command-line tool.

5. Use the asadmin deploy command to deploy the WAR module.

   For example,

   ```
   asadmin> deploy --user admin --password admin --host localhost --port 4848 --type web --
   instance server1 /install_dir/samples/webservices/jaxrpc/simple/Hello.war
   ```

   For more information on using the asadmin command-line tool, see the *Sun Java System Application Server Developer's Guide*.

Invoking JAX-RPC Web Services

Invoking a Web service essentially refers to the actions that a client application performs to access a Web service. Web services deployed to Sun Java System Application Server can be accessed by any client. That is, any J2EE component within the application server can take the role of a client. Any application or an application client can access Web services. A client can use Apache SOAP libraries to make a call to a Web service or it could be a .net client.

This section describes the procedures to develop JAX-RPC clients that can invoke JAX-RPC Web services deployed to Sun Java System Application Server.

JAX-RPC clients are applications that use the JAX-RPC APIs and runtime for invoking a Web service. These clients import the service using WSDL and can invoke a service that has been defined and deployed on a non-Java platform. JAX-RPC defines the `javax.xml.rpc.Service` interface to model the Web service from a client's perspective. You can use either J2SE or J2EE client programming model to develop JAX-RPC clients.

The main steps in invoking a Web service are listed below:

1. Add the Java client JAR files to the client jar path. For more information on how to add the jar files to the classpath, see Setting Up the Client Environment.
2. Create a Java-based service client.
3. Assemble and deploy your client application. See Assembling and Deploying a JAX-RPC Client.
4. Execute your Java client to invoke the Web service.

You can create JAX-RPC clients using the stubs method, a dynamic proxy, or the call interface method. This section discusses the following topics:

- Creating Clients Using Generating Stubs Method
- Creating Clients Using Dynamic Invocation Interface
  - Creating JAX-RPC Client Using a Dynamic Proxy
  - Creating a JAX-RPC Client Using the Call Interface
- Assembling and Deploying a JAX-RPC Client

Creating Clients Using Generating Stubs Method

Stubs are used when a JAX-RPC client knows what method to call and how to call it, such as what parameters to pass. Invoking a remote method through a stub is like invoking a remote method using the Java Remote Method Invocation (RMI) system. A stub simplifies the remote method calls by making them appear like local method calls. A local stub object is used to represent a remote object. To make a remote method call, a JAX-RPC client makes the method call on the local stub.

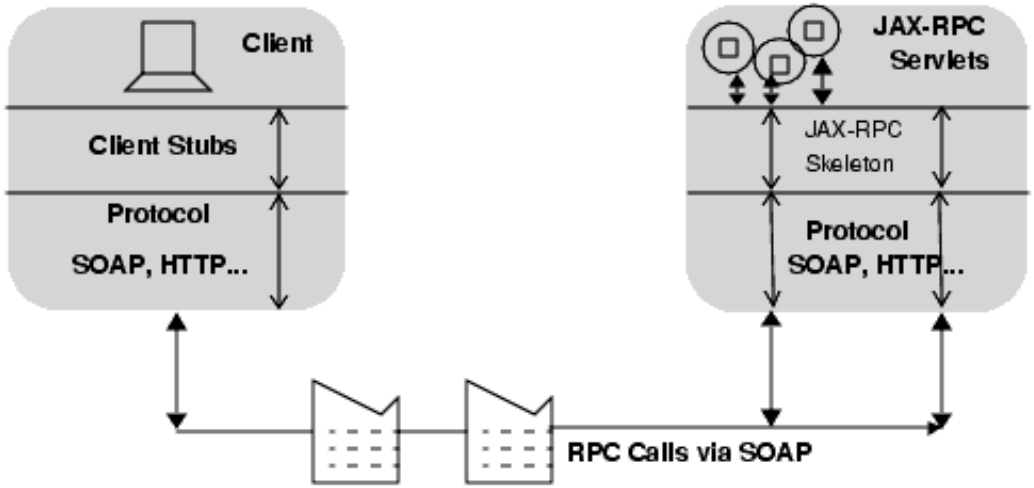
A stub class is a mapping of a port in the WSDL that describes the Web service. It must therefore implement the service definition interface that reflects the methods of the associated `portType`. Thus the client has strongly typed, early-bound access to the Web service endpoint.

The stub must also implement the `javax.xml.rpc.Stub` interface, which provides the facility for the client to configure the stub dynamically.

Typically, a JAX-RPC client performs the following steps. These steps are illustrated in the figure JAX-RPC Client Model.

1. The client calls the stub.
2. The stub redirects the call to the appropriate Web service.
3. The server catches the call and redirects it to a framework.
4. The framework wraps the actual implementation of the service, then calls the Web service on behalf of the client.
5. The framework returns the call to the server.
6. The Web service, in turn, returns the information to the originating client stub.
7. Finally, the client stub returns the information to the client application.

JAX-RPC Client Model



The following sections describe these steps:

- Generating the Stubs
- Coding the Client
- Compiling the Client Code
- Assembling the Client Classes into a JAR file
- Running the Client

Generating the Stubs

You can use the `wscompile` tool to generate the stubs for the client. In addition to generating the stubs, the `wscompile` tool also

generates the ties for the server. To generate stubs, set the PATH to the *install_dir*`/share/bin` directory in case of file-based installation of the application server and `usr/share/bin` directory in case of package-based installation of the application server. Run the tool using the following syntax:

```
wscompile -gen:client -d build/client -classpath build/shared config.xml
```

For more information on `wscompile` tool, see wscompile Tool.

There are two ways to generate the stubs. The stubs can be generated from the service endpoint definition or from a WSDL document. The `wscompile` command above uses the service endpoint definition to generate the stubs.

Coding the Client

Make sure to add the necessary jar files to the classpath. For more information, see Setting Up the Client Environment.

The client performs the following steps:

1. Obtain an instance of the interface stub.
2. Set the endpoint property of the stub to point to the service endpoint of the Web service.
3. Call the method.

In this example, the stubs are generated using the service endpoint definition. You provide the configuration information using the JAX-RPC implementation's client-side API `javax.xml.rpc.Stub`.

The following code illustrates the above mentioned steps:

```
package hello;

import javax.xml.rpc.Stub;

public class HelloClient {
public static void main(String[] args) {
try {

HelloIF_Stub stub = (HelloIF_Stub)(newHelloWorld_Impl().getHelloIFPort());

stub._setProperty( javax.xml.rpc.Stub.ENDPOINT_ADDRESS_PROPERTY, args[0]);
```

```
System.out.println(stub.sayHello("Duke!"));

command-line  }

catch (Exception ex) {

ex.printStackTrace();

     }

  }

}
```

In the code illustration above, the `HelloClient` is a stand-alone program that calls the `sayHello` method of the `HelloWorld` service. It makes this call through a stub, a local object that acts as a proxy for the remote service. In the code listing, note the names of the `HelloIF_Stub` and `HelloWorldImpl` classes, which were generated by the `wscompile` tool. The `HelloIF` prefix matches the name of the service definition interface and the `HelloWorld` prefix corresponds to the service name specified in the configuration file. The `HelloWorldImpl` class is the implementation of a service as described in the JAX-RPC specification. The client gets a reference to the stub by calling the `getHelloIF` method of the `HelloWorldImpl` class, which was created when you ran the `wscompile` tool.

The args[0] parameter of the `stub._setProperty` method is a URI that denotes the address of the target service port.

Compiling the Client Code

Note Be sure to run the `wscompile` and the `wsdeploy` tools before you compile the client code. The client code refers to the generated by the `wscompile` tool.

```
wscompile -gen:client -d build/client -classpath build/shared config.xml
```

To compile the client, go to the directory where you have the client code saved and type the following command:

```
asant compile-client
```

This command compiles the Java source code.

Assembling the Client Classes into a JAR file

You can use the `asant` tool to assemble the client classes into a JAR file. `asant` is a command-line interface tool. Type the following command:

```
asant create-client-jar
```

This command creates the client jar file.

Running the Client

Use the `asant` tool to run the client. Type the following command:

```
asant run
```

The `run` target of `asant` executes this command:

`java -classpath` *cpath* `client` *endpoint-address*

cpath - The classpath includes the client jar file that you have created, as well as several other JAR files that are part of the JAX-RPC implementation.

Note In order to run the client remotely, all of these JAR files must reside on the remote client system.

---

endpoint-address - The endpoint-address is the URL which the stub uses to access the Web service. The endpoint-address is set when you are coding the client, using the following codeline:

```
stub._setProperty
```

```
(javax.xml.rpc.Stub.ENDPOINT_ADDRESS_PROPERTY, args[0]);
```

At runtime, the endpoint is passed to the client as a command-line parameter.

For example: `http://localhost:8080/jaxrpc-hello/jaxrpc`

The `jaxrpc-hello` portion of the URL is the context of the servlet that implements the `HelloWorld` service. This portion corresponds to the prefix of the `jaxrpc-hello.war` file.

You can accomplish the task of compiling, assembling and deploying, and running a client through a `build.xml` file. The `build.xml` file for the sample bundled with Sun Java System Application Server is available at the following location:

install_dir`/samples/webservices/jaxrpc/simple/src`

Creating Clients Using Dynamic Invocation Interface

Using Dynamic Invocation Interface (DII), a client can call a service or a remote procedure. The client can discover the name of the service or the procedure at runtime, making use of a service broker that can dynamically look up the service and its remote procedures.

The `javax.xml.rpc.Service` encapsulates two types of dynamic invocation that do not require any generated code. This section describes the procedure to create dynamic clients.

- Creating JAX-RPC Client Using a Dynamic Proxy
- Creating a JAX-RPC Client Using the Call Interface

Creating JAX-RPC Client Using a Dynamic Proxy

A JAX-RPC client can interact with a Web service using a dynamic proxy. A dynamic proxy is a class that dynamically supports service endpoints at runtime, without having pre generated stubs. A client creates dynamic proxies by calling the `getPort()` method of the `javax.xml.rpc.Service` interface. The client calls its `getPort()` method, passing in the Java service definition interface and the corresponding Web service port name. It passes back a dynamically built and configured implementation of the service definition interface-a dynamically built stub.

For more information on dynamic proxies, visit the following URL:
http://java.sun.com/j2se/1.3/docs/guide/reflection/proxy.html

The steps given below explains the procedure to create a dynamic proxy client.

1. Make sure to add necessary jar files to the classpath. For more information on adding jar files to the classpath, see Setting Up the Client Environment.
2. Create a client class that uses a dynamic proxy to invoke the service.

```
public class HelloClient {
....
....
}
```

3. Define the name of the service, the port name and the name of the WSDL that contains the information about the Web service you wish to access.

```
String UrlString = endpoint;
String nameSpaceUri = "http://proxy.org/wsdl/HelloWorld";
String serviceName = "HelloWorld";
String portName = "HelloIFPort";
URL helloWsdlUrl = new URL(UrlString);
```

4. Obtain an instance of the default implementation for `ServiceFactory` object.

```
ServiceFactory serviceFactory = ServiceFactory.newInstance();

Service helloService =serviceFactory.createService(helloWsdlUrl, new
QName(nameSpaceUri,serviceName));
```

5. Get a dynamic proxy for the object.

```
HelloIF myProxy =(HelloIF)helloService.getPort( new QName(nameSpaceUri,portName),
proxy.HelloIF.class);
```

   In the code illustration, the `getPort()` method is passed in an interface definition that will be used as a template for building a runtime instance of a dynamic proxy.

6. Invoke the service using `java.lang.reflect.InvocationHandler.Invoke();`

```
System.out.println(myProxy.sayHello("Buzz"));
```

Creating a JAX-RPC Client Using the Call Interface

In the Call Interface approach, a client dynamically discovers services, configures the remote calls, and executes the calls.The client uses the `javax.xml.rpc.Call` interface for the dynamic invocation of a JAX-RPC service. At runtime, the client uses the DII to call remote procedures on the Web service.

DII `Call` object method supports two types of invocation, namely, synchronous request-response and one-way mode. In the synchronous request-response mode, the client uses the invoke method of the call object to make a remote method. The client then waits until the operation is complete, that is, until a response is returned. In one-way method, the client uses the `invokeOneWay` method of the call object to make a remote call.

The steps given below explains the procedure to create a client that can invoke a Web service using the call interface approach:

1. Make sure to include the necessary jar files to the classpath. For more information, see Setting Up the Client Environment.
2. When you create a dynamic client, define the name of the service that you wish to access and the port name. Then, you create a service factory using the `ServiceFactory.newInstance()` method. The `ServiceFactory.newInstance()` method is supported by the JAXR API to define a service . For more information, see Adding Services and Service Bindings to an Organization.

   ```
   private static String qnameService = "HelloWorld";
   private static String qnamePort = "HelloIF";

   ServiceFactory factory = ServiceFactory.newInstance();

   Service service = factory.createService(new QName(qnameService));
   ```

3. Create a `Service` object from the factory.

   ```
   Service service = Factory.createService(new QName(qnameService));
   ```

4. Create a `Call` object from the `service` and pass the name of the port and the operation you want to execute.

   ```
   QName port = new QName(qnamePort);

   Call call = service.createCall();
   call.setPortTypeName(port); call.setTargetEndpointAddress(endpoint);
   ```

5. Set the property prior to making the actual method call. The `setProperty` method is used to set standard properties that are listed in the JAX-RPC specification.

```
call.setProperty(Call.SOAPACTION_USE_PROPERTY, new       Boolean(true));

call.setProperty(Call.SOAPACTION_URI_PROPERTY, "");
call.setProperty(ENCODING_STYLE_PROPERTY, URI_ENCODING);
QName QNAME_TYPE_STRING = new QName(NS_XSD, "string");
call.setReturnType(QNAME_TYPE_STRING);
```

6. Set the operation name.

```
call.setOperationName(new   QName(BODY_NAMESPACE_VALUE,"sayHello"));
call.addParameter("String_1", QNAME_TYPE_STRING, ParameterMode.IN);
String[] params = { new String("Duke!") };
```

   The `addParameter` method is used to add a parameter and the type for the operation specified in the `setOperationName` method. Note that the values of these parameters are obtained from the WSDL document for the service.

7. Use the `Call.invoke()` method to invoke the service.

```
String result = (String)call.invoke(params);
```

   The `invoke` method invokes the operation specified in the `setOperationName` method using a synchronous request-response interaction mode. The method call specifies the input parameters for the invocation.

Creating a JAX-RPC Client Using a WSDL

The following steps describes the procedure to create a dynamic client that uses the WSDL to locate a Web service and invoke the service.

1. Create a service factory using the `ServiceFactory.newInstance()` method.

```
ServiceFactory serviceFactory = ServiceFactory.newInstance();
```

2. Create a `service` object from the factory. Pass the name of the WSDL.

```
String nameSpaceUri = “http://hello.org/wsdl”;

URL helloWsdlURL = new URL(URLstring);
```

3. Create a `serviceName` object and pass the name of the service that you wish to invoke.

   ```
   String serviceName = "HelloWorld";
   ```

4. Create a `portName` object and specify the port name.

   ```
   String portName = "HelloIFport";
   ```

5. Create an `OperationName` object and specify the name of the operation in the service that you wish to execute.

   ```
   String operationName = "sayHello";
   ```

6. Create a service, passing it the WSDL location and the name of the service that you want to invoke.

   ```
   Service helloService = serviceFactory.createService(helloWsdlURL, new
   QName(nameSpaceUri, serviceName"));
   ```

7. Create a `Call` object, pass it the name of the port and the operation that you want to execute.

   ```
   Call call = helloService.createCall(portName, operationName);
   ```

8. Invoke the service using the `Call.Invoke()` method.

   ```
   String result = String call.Invoke(helloService);
   ```

Assembling and Deploying a JAX-RPC Client

JAX-RPC Clients can be bundled into a deployable WAR file using the `wsdeploy` command tool. The `wsdeploy` command reads the JAX-RPC runtime descriptor `jaxrpc-ri.xml` file and the web application deployment descriptor `web.xml` file. Assembling and deploying a JAX-RPC client involves the following steps:

1. Create the JAX-RPC runtime descriptor file. The name of the file must be `jaxrpc-ri.xml`. See The jaxrpc-ri.xml File.
2. JAX-RPC client is a web module. Create a web module deployment descriptor `web.xml`. For information on `web.xml` file, see the *Sun Java System Application Server Developer's Guide to Web Applications*.
3. Use the `wsdeploy` command tool to create a deployable WAR file. For information about `wsdeploy` command tool, see wsdeploy Tool.
4. Deploy the WAR file using asadmin deploy command.

You can accomplish the task of assembling and deploying, and running a JAX-RPC client through an ant `build.xml` file. The `build.xml` file for the samples are bundled with Sun Java System Application Server which is available at the following location:

install_dir`/samples/webservices/jax-rpc/simple/src`

Sample Applications

- *install_dir*`/samples/webservices/jaxrpc/proxy` - contains a sample dynamic proxy client application that illustrates the basics of creating, deploying, and accessing a Web service.
- *install_dir*`/samples/webservices/jaxrpc/dynamic` - contains a dynamic invocation interface client that illustrates the basics of creating, deploying, and accessing a Web service.

JAX-RPC Client Invoking an EJB

This section describes the procedure to create a stand-alone JAX-RPC client that makes a remote call on a JAX-RPC service. This service locates a stateless session bean and invokes a method on the bean.

Note These instructions apply to the development of JAX-RPC services only in the J2EE 1.3.1 environment.

The main steps to invoke an EJB are listed below:

1. Create a stateless session bean. See the *Sun Java System Application Server Developer's Guide to Enterprise Java Beans* for detailed instructions on creating a stateless session bean.
2. Create a JAX-RPC Web service that performs a lookup on the EJB. The following code illustrates how a Web services application can make a remote call on the EJB:

```
public String sayHello(String name) {

Context initial = new InitialContext();
Context myEnv = (Context)initial.lookup("java:comp/env");
Object objref = myEnv.lookup("ejb/SimpleGreeting");
```

```
GreetingHome home =
(GreetingHome)PortableRemoteObject.narrow(objref,GreetingHome.class);
```

3. Create a stand-alone client that makes a remote call on the JAX-RPC service. The following code is an example of the client that makes a remote call on the EJB.

```
package samples.webservices.jaxrpc.toejb.client;

import javax.naming.Context;
import javax.naming.InitialContext;
import javax.rmi.PortableRemoteObject;
import samples.webservices.jaxrpc.toejb.ejb.*;

public class GreetingClient {

  public static void main(String[] args) {
try {

Context initial = new InitialContext();
Context myEnv = (Context)initial.lookup("java:comp/env");
Object objref = myEnv.lookup("ejb/SimpleGreeting");

GreetingHome home =
(GreetingHome)PortableRemoteObject.narrow(objref,GreetingHome.class);

Greeting salutation = home.create();

System.out.println(salutation.sayHey("Buzz"));

System.exit(0);

} catch (Exception ex) {

System.err.println("Caught an unexpected exception!"); ex.printStackTrace(); }

} // main
```

You can find the complete code listing for the sample at: *install_dir*/samples/webservices/jax-rpc/toejb/src/

4. Assemble the service and the client. See Assembling and Deploying JAX-RPC Web Services and Assembling and Deploying a JAX-RPC Client.
5. Deploy the session bean by performing the following steps:
    - Edit the deployment descriptor files (`ejb-jar.xml` and `sun-ejb-jar.xml`).
    - Execute an Ant build command (such as build jar) to reassemble the JAR module.
    - Use the `asadmin` deploy command to deploy the JAR module. For example, the following command deploys an EJB application as a stand-alone module:

      ```
      asadmin deploy --type ejb --instance inst1 myEJB.jar
      ```

6. Deploy the JAX-RPC service. See Assembling and Deploying JAX-RPC Web Services.
7. Run the JAX-RPC client using the asant command.
   ```
   asant run
   ```

Building Security into JAX-RPC Web Services

This section describes the procedure to provide security to a JAX-RPC service application, by providing security to the web container that contains the application, using HTTP/SSL for basic and mutual authentication. For more information on authentication, see the *Sun Java System Application Server Developer's Guide to Web Applications*.

This section presents the following topics:

- Basic Authentication Over SSL
- Adding Security Elements to web.xml
- Setting Security Properties in the Client Code
- Mutual Authentication Over SSL
- Setting Up Client Certificate Authentication for Web Services

You must perform the following steps to configure a JAX-RPC Web service endpoint for HTTP/S basic and mutual authentication:

- Use `keytool`, which is part of the J2SE SDK, to generate certificates and keystrokes.
- Add security elements to `sun-web.xml` deployment descriptor.
- Add some properties in the client code.
- Build and run the Web service.

Basic Authentication Over SSL

Follow the steps below to configure a Web service for basic authentication over HTTP/S:

1. Configure a certificate and enable SSL on HTTP listener for your server. For more information on configuring a certificate and enabling SSL on HTTP, see “Administering Certificates” and “Turning Security On” sections respectively, in the *Sun Java System Application Server Administrator’s Guide to Security*.

   A HTTP client uses a repository of trusted Certificate Authorities (CA) during the SSL handshake to validate server certificate. It is important that the CA of your server certificate be a trusted CA for the client.

2. For the J2SE 1.4 based clients which includes JSSE based clients, such as Web services applications, you need to import the certificate of your server’s CA into JSSE cacerts database. Use the command line tool, the `keytool` to import the trusted CA certificate.

   The following code illustrates how you can import the certificate of your CA into the trusted CA database of your J2SE-based client:

   ```
   keytool -import -v -alias "CMS-CA" -file cmsca.cer -keystore cacerts
   ```

   For more information on using `keytool`, run `keytool` command with `-help` option or visit the following URL:

   http://java.sun.com/j2se/1.3/docs/tooldocs/solaris/keytool.html

3. Enter keystore password: `changeit`.

   ```
   Owner: CN=Certificate Manager, OU=AppServices, O=Sun Microsystems, L=SCA, ST=California,
   C=US

   Issuer: CN=Certificate Manager, OU=AppServices, O=Sun Microsystems, L=SCA,
   ST=California, C=US

   Serial number: 1

   Valid from: Mon Jun 03 12:00:00 PDT 2002 until: Thu Jun 03 12:00:00 PDT 2004

   Certificate fingerprints:

   MD5: 6C:8D:A6:E4:55:52:1A:FF:9D:19:44:D7:0F:62:66:95
   ```

```
SHA1:89:B1:0E:7E:8F:56:B2:34:65:46:15:86:53:7E:3E:6B:4F:9D:84:63

Trust this certificate? [no]: yes

Certificate was added to keystore

[Saving cacerts]
```

4. Configure the server instance to use appropriate realm and make sure the realm has users you would like to permit the Web services access.

   For Example, to set up the flat file of users, follow the steps below in the Administration interface:

   - Select the server instance and click on the Security node in the left pane.
   - Select the drop-down box for the Default Realm in the right pane and choose the option "file".
   - Select the Realms in the left pane and click on the file realm to add the users to the file realm.
   - Apply your changes. Now, the server's flat file user database is ready to use.

     For detailed information on Configuring the server instance to use the realm, see the *Sun Java System Application Server Administration Guide*.

Adding Security Elements to web.xml

Enable Basic Authentication for the web application and specify a security constraint to enforce authentication. For more information on security elements in `web.xml`, see the *Sun Java System Application Server Developer's Guide to Web Applications*.

Here is an example of how you can configure the Basic Authentication for the Web service servelt-based end point. This security-constraint allows principals with the role "ServiceUser" which is mapped to user "bob" in the `sun-web.xml`.

The `WEB-INF/web.xml`:

```
<security-constraint>

  ....
  <web-resource-collection>
    <web-resource-name>SecureHello</web-resource-name>
    <url-pattern>/security</url-pattern>
    <http-method>GET</http-method>
```

```
    <http-method>POST</http-method>
  </web-resource-collection>

  <auth-constraint>
    <role-name>manager</role-name>

    </auth-constraint>

</security-constraint>

<login-config>

  <auth-method>BASIC</auth-method>

</login-config>
```

The `WEB-INF/sun-web.xml`:

```
<sun-web-app>
<security-role-mapping>

  <role-name>ServiceRole</role-name> <principal-name>bob</principal-name>

</security-role-mapping>
</sun-web-app>
```

Next, set the security properties for the J2SE-based client. For step-by-step instructions, see Setting Security Properties in the Client Code.

Setting Security Properties in the Client Code

For basic authentication over SSL, the client code must set several security-related properties.

trustStore Property

The client specifies the `trustStore` property as follows:

```
System.setProperty("javax.net.ssl.trustStore", trustStore);
```

trustStorePassword Property

The `trustStorePassword` property is the password of the J2SE SDK keystore. In the previous section, you specified the default password `changeit` when running `keytool`. The client sets the `trustStorePassword` property in the following code illustration:

```
System.setProperty("javax.net.ssl.trustStorePassword", trustStorePassword);
```

Username and Password Properties

The *username* and *password* values correspond to the manager role. The client sets the `username` and `password` properties as follows:

```
stub._setProperty(javax.xml.rpc.Stub.USERNAME_PROPERTY, username);
```

stub._setProperty(javax.xml.rpc.Stub.PASSWORD_PROPERTY, password);

Mutual Authentication Over SSL

To configure and create a JAX-RPC service with mutual authentication, follow all of the steps in the Basic Authentication Over SSL. Then, follow these steps:

1. Export the generated client certificate.

   The following code illustrates how to export the client certificates.

   ```
   $Java_home/bin/keytool -export -alias -client -storepass changeit -file client.cer -
   keysnttore client.keystore
   ```

2. Import the client certificate into the server's keystore.

   ```
   $Java_home/bin/keytool -import -v -trustcacerts -alias -client -file client.cert -
   keystore server.keystore -keypass changeit -storepass changeit
   ```

3. Run the application using `asant` tool.

   ```
   asant run
   ```

Setting Up Client Certificate Authentication for Web Services

This section describes how you can configure a Web service that uses Client Certificate Authentication with Sun Java System Application Server.

Note For clients, using HTTPS, follow the steps 1-6 given below.

---

1. Follow the steps 1 and 2 described under Basic Authentication Over SSL.
2. For J2SE-based clients, generate a key-pair for the client certificate using the keytool. The key-pairs are stored in a keystore. The following code line illustrates how to generate a key-pair:

   ```
   > keytool -genkey -v -alias clcert -dname 'CN=Test User, OU=testOU, O=testO, L=SCA,
   S=California, C=US'\

     -keystore clcerts -keypass changeit
   ```

   The command execution, displays the following information:

   ```
   Generating 1,024 bit DSA key pair and self-signed certificate (SHA1WithDSA)

   for: CN=Test User, OU=testOU, O=testO, L=SCA, ST=California, C=US

   Enter key password for <clcert1>

     (RETURN if same as keystore password):
   [Saving clcerts]
   ```

3. Generate a certificate request to be sent to a trusted CA. Use the following `keytool` command to generate a certificate request:

   ```
   >keytool -certreq -v -alias clcert -file clreq -keystore clcerts
   ```

   Enter keystore password: `changeit`

   Certification request stored in the file: `clreq`

   Submit this to your CA

```
>more clreq
```

This command generates the following message:

```
-----BEGIN NEW CERTIFICATE REQUEST-----

MIICazCCAigCAQAwZTELMAkGA1UEBhMCVVMxEzARBgNVBAgTCkNhbGlmb3JuaWExDDAKBgNVBAcT
A1NDQTEOMAwGA1UEChMFdGVzdE8xDzANBgNVBAsTBnRlc3RPVTESMBAGA1UEAxMJVGVzdCBVc2Vy

..

GqdqJvx+mVcgcQkdzap+ykIEcOZ/qQq60rPddF6yK9wnWkez32Y2btGWe598oAAwCwYHKoZIzjgE
AwUAAzAAMC0CFQCPf7TuJLJ+zS/HH+fCcKnFAtmKYwIUZM10c56KrScgledImxGzLIKMsGE=

-----END NEW CERTIFICATE REQUEST-----
```

4. Send the generated certificate request to a trusted CA and request a signed certificate. For more information on sending the request to a trusted CA, see *CA Server Administration* documentation.

   You may verify the returned signed certificate in Base64 encoded X.509 format (saved for example in `calcert.txt` file). Note that this certificate is chained as it is signed by the CA. If you use a pkcs7 format, the `keytool` may complain that the certificate us not in X.509 format.

   ```
   > keytool -printcert -v -file clcert.txt

   Certificate[1]:

   Owner: CN=Certificate Manager, OU=AppServices, O=Sun Microsystems, L=SCA, ST=Californa,
   C=US

   Issuer: CN=Certificate Manager, OU=AppServices, O=Sun Microsystems, L=SCA, ST=Califoria,
   C=US

   Serial number: 1

   Valid from: Mon Jun 03 12:00:00 PDT 2002 until: Thu Jun 03 12:00:00 PDT 2004

   Certificate fingerprints:
   ```

```
MD5: 6C:8D:A6:E4:55:52:1A:FF:9D:19:44:D7:0F:62:66:95
SHA1: 89:B1:0E:7E:8F:56:B2:34:65:46:15:86:53:7E:3E:6B:4F:9D:84:63

Certificate[2]:
Owner: CN=Test User, OU=testOU, O=testO, L=SCA, ST=California, C=US
Issuer: CN=Certificate Manager, OU=iWSQA, O=Sun Microsystems, L=SCA, ST=Califoria, C=US

Serial number: 19

Valid from: Thu Sep 12 18:33:54 PDT 2002 until: Fri Sep 12 18:33:54 PDT 2003

Certificate fingerprints:

MD5: 82:09:8A:DC:E2:85:82:B5:56:98:93:81:97:A9:D5:32
SHA1: 1D:7C:F2:F2:ED:79:A3:62:0A:A2:1B:22:74:11:BF:52:CB:8D:9E:BB
```

5. Update the keystore with the signed client certificate.

   ```
   > keytool -import -v -trustcacerts -alias clcert -file clcert.txt -keystore clcerts -
   keypass changeit
   ```

   Displays the following message:

   ```
   Certificate was added to keystore

   [Saving clcerts]
   ```

6. Now, you must set up the client Java virtual machine to use this keystore/password:`-Djavax.net.ssl.keyStore=<path-to/clcerts>` and `-Djavax.net.ssl.keyStorePassword`.

   If you are running the client within `<java>` ant target, you can use the `<sysproperty>` element to specify the keystore/password.

   For example:

   ```
   <sysproperty key="javax.net.ssl.keyStore"

   value="C:/security/clcerts"/>
   ```

```
<sysproperty key="javax.net.ssl.keyStorePassword" value="changeit"/>
```

7. Enable the certificate realm for the server instance.
   - In the Administration interface, select the server instance in the left pane.
   - Click the Security node and choose Default Realm to Certificate.

   For more information on enabling the certificate alarm, see the *Sun Java System Application Server Administration Guide*.

8. Edit the `web.xml` deployment descriptor to configure the Web service application to use CLIENT-CERT authentication.

```
security-constraint>

<web-resource-collection>

    <web-resource-name>Protected Area</web-resource-name>
<url-pattern>/*</url-pattern>
<http-method>GET</http-method>
<http-method>POST</http-method>

</web-resource-collection>
<auth-constraint>

      <role-name>TesterRole</role-name>

</auth-constraint>

</security-constraint>

<login-config>

<auth-method> CLIENT-CERT </auth-method>

</login-config>
```

9. Edit the Sun Java System Application Server specific deployment descriptor (`sun-web.xml`) to map the role to the X.509 principal name DN of the client certificate.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE sun-web-app PUBLIC '-//Sun Microsystems, Inc.//DTD Sun Java System Application
Server 7.0 Servlet 2.3//EN'

'http://www.sun.com/software/appserver/dtds/sun-web-app_2_3-0.dtd'

<sun-web-ap>

<security-role-mapping>

<role-name>TesterRole</role-name>

<principal-name>CN=Test User, OU=testOU, O=testO, L=SCA, ST=California, C=US</principal-
name>

</security-role-mapping>

</sun-web-app>
```

10. Alternatively, you may want to set an optional assign-groups property that can be set for the certificate realm configuration to which all certificate users belong. This will allow you to do the role-mapping to this group name, instead of having to list the principal name DNs.
    - In the Administration interface, select the server instance in the left pane.
    - Click and expand the Security node and the Realms node.
    - Click on the certificate realm and on the right pane, click on the properties link to add the property names `cert-users` with the value assign-groups.
    - Save and Apply the changes. The server.xml reflects the configuration settings:

```
<security-service>

<auth-realm name="certificate"
classname="com.iplanet.ias.security.auth.realm.certificate .CertificateRealm">

<property value="cert-users" name="assign-groups"/>

</auth-realm>

</security-service>
```

For more information on configuring the certificate realm, see the *Sun Java Application Server Administration Guide*.

11. Restart the server and run the client to verify the working of client certificate authentication.

JAX-RPC Tools

JAX-RPC implementation of Sun Java System Application Server includes the following tools that helps in the development of JAX-RPC clients.

- wscompile Tool
- wsdeploy Tool

wscompile Tool

`wscompile` is a mapping tool that is bundled with Sun Java System Application Server which generates stubs, ties, serializers, and other artifacts. You can also use this tool to generate a WSDL document from the service endpoint definition or produce a service endpoint definition from the WSDL document.

You have the option of generating only the client-side artifacts such as stubs, server-side artifacts such as ties, or both client and server-side artifacts. The tool reads the configuration file that contains information needed to generate the artifacts.

The syntax of the `wscompile` command is as follows:

```
wscompile [options] config_file_name
```

wscompile Command Options

The following table lists the options that you can use with the `wscompile` command. The first column specifies the option that you can use with the command, and the second column describes the option.

Table 2-1 wscompile Tool Options

Option	Description

`-gen:client`	Generates client-side artifacts such as stubs, service interface, implementation classes, and remote interface.
`-gen:server`	Generates server-side artifacts such as ties, server configuration file, WSDL file, service definition interface. If you are using `wsdeploy` tool, you must not use this option.
`-gen:both`	Generates both client and server-side artifacts at the same time.
`gen`	Same as gen:client.
`-define`	Defines a service.
`-classpath<classpath_string>`	Specify the path of input class files.
`cp<classpath_string>`	Same as classpath.
`-d<directory_name>`	Sets the output directory for all generated files.
`-s<directory>`	Specifies the path where the generated files will be stored.
`-f:<features>/-features:`	Enable the listed features. Features are separated by comma. List of features supported are:

<features>	1. datahandleronly - always map the attachments to data handler types 2. explicitcontext - turn on explicit service mapping context 3. infix=<name> - Specify an infix to use for generated serializers 4. nodatabindings - turn off data bindingss for literal encoding 5. noencodedtypes - turn off encoding type information 6. nomultirefs - turn off support for multiple references 7. novalidation - turn off validation for imported WSDL file 8. searchschema - search schema aggressively for subtypes 9. serializeinterfaces - turn on direct servialization of interface type
-g	Generates debugging info.
-httpproxy:<host>:<port>	Specify HTTP proxy server.
-import	Generate interfaces and value types.
-keep	Keep the generated .java files after the compilation is complete.
-model<file>	Write internal model to the file.
-nd<directory>	Specify the path to store non class generated files.

-O	Optimize the generated code.
-verbose	Output messages about the compiler action.
-version	Print version information.

You must use the wscompile command with one of the -import, -gen, and -define options. Invoking wscompile command with no options will display the usage information.

Configuration File

The wscompile tool reads the configuration file, which contains information that describes the Web service. The basic structure of the configuration file is given below:

```
<?xml version="1.0" encoding="UTF-8"?>

<configuration

xmlns="http://java.sun.com/xml/ns/jax-rpc/ri/config">
  <service> or <wsdl> or <modelfile>

</configuration>
```

If you use the <service> element in your configuration file, the wscompile tool reads the RMI interface that describes the service and generates a WSDL file.

If you use the <wsdl> element in your configuration file, the wscompile tool reads the service's WSDL file and generates the service's RMI interface.

If your configuration file contains a <service> or <wsdl> element, the wscompile tool generates a model file that contains the internal data structure that describe the service. If you have already generated a model file in this manner, then you can reuse it the next

time you run the `wscompile` tool.

For information on the XML schema to create a configuration file, see Appendix A, "XML Schema Definitions."

Configuration File with RMI Interfaces

```
<?xml version="1.0" encoding="UTF-8"?>

<configuration xmlns="http://java.sun.com/xml/ns/jax-rpc/ri/config">

<service name="[1]"

  targetNamespace="[2]"
  typeNamespace="[3]">
  packageName="[4]">

    <interface name="[5]"

servantName="[6]"
      soapAction="[7]"
      soapActionBase="[8]"/>

<typeMappingRegistry> [9] </typeMappingRegistry>

</service>
</configuration>
```

The configuration file contains the following Web services attributes:

- `servicename` - This attribute is used to generate a properties file that the servlet-based JAX-RPC runtime uses for dispatching the request to tie-and-servant combination.
- `targetNamespace` - This attribute specifies the target name space for the generated WSDL document.
- `typeNamespace` - This attribute specifies the target name space for the schema portion of the generated WSDL document.
- `packageName` - Specifies the package name for the generated Java classes. For example, the service interface extending `javax.xml.rpc.Service`.
- `interface name` - Specifies the fully qualified name of a Java interface.
- `servantName` - Speicifies the fully qualified name of a servant class.
- `soapAction` - String used as the `SOAPAction` for all operations in the corresponding port. This is optional.

- `soapActionBase` - String used as a prefix for the `SOAPAction` strings for the operations in the corresponding port.
- `typeMappingRegistry` - Specifies the type mapping information.

Note One generic servlet class `com.sun.xml.rpc.server.http.JAXRPCServlet` is used for all JAX-RPC endpoints.

Configuration file with a WSDL document

```
<?xml version="1.0" encoding="UTF-8"?>

<configuration xmlns="http://java.sun.com/jax-rpc-ri/xrpcc-config">

<wsdl location="[1]"

packageName="[2]">
<typeMappingRegistry>[3] </typeMappingRegistry>

</wsdl>

</configuration>
```

The configuration file with a WSDL document has the following attributes:

- `wsdl location` - URL pointing to aWSDL document.
- `packageName` - Specifies a fully qualified name of the Java package for the generated classes/interfaces.
- `typeMappingRegistry` - type mapping information.

wsdeploy Tool

The `wsdeploy` tool generates a deployable WAR file for a service.This tool takes as input a WAR file for the service.

Syntax of the `wsdeploy` command is as follows:

`wsdeploy` *[options] war file*

wsdeploy Command Options

The following table lists the options that you can use with the `wsdeploy` command. The first column lists the options, and the second column describes the option.

Table 2-2 wsdeploy Tool Options

Option	Description
`-classpath<classpath_string>`	Specify the path of input class files.
`cp<classpath_string>`	Same as classpath.
`-tmp<directory_name>`	Specify the path of the temporary directory.
`-o<output war file>`	Specify the path where the generated WAR file will be stored. This option is required.
`-keep`	Keep the generated .java files after the compilation is complete.
`-verbose`	Output messages about the compiler action.
`-version`	Print version information.

`war file`- Typically, you create the WAR file with a development tool or with the `asant` war task. The following are the contents of a simple WAR file:

```
META-INF/MANIFEST.MF
WEB-INF/classes/hello/HelloIF.class
WEB-INF/classes/hello/HelloImpl.class
WEB-INF/jaxrpc-ri.xml
WEB-INF/web.xml
```

In the example, `HelloIF` is the service's RMI interface and `HelloImpl` is the class the implements the interface. The `web.xml` file is the deployment descriptor of a web component.

The `wsdeploy` tool examines the deployment descriptor `web.xml` and `jaxrpc-ri.xml` to generate the WAR file. If the deployment descriptor identifies a model file, the information in the model file is used for generating a WAR file. If the deployment descriptor does not identify a model file, wsdeploy generates a model. For information about the XML schema for creating a model file, see Appendix A, "XML Schema Definitions."

Behind the scene, `wsdeploy` tool runs the `wscompile` tool with `-gen:server` option. In other words, the tool generates the server-side artifacts such as ties. This tool can also generate the service endpoint definition, or a WSDL document.

The jaxrpc-ri.xml File

The `jaxrpc-ri.xml` file is the JAX-RPC implementation specific configuration file. This configuration file is read by the `wsdeploy` tool. The following code lists the contents of a `jaxrpc-ri.xml` file for a simple `HelloWorld Service`.

```
<?xml version="1.0" encoding="UTF-8"?>

<webServices

xmlns="http://java.sun.com/xml/ns/jax-rpc/ri/dd"
  version="1.0
  targetNamespaceBase="http://com.test/wsdl"
  typeNamespaceBase="http://com.test/types"
  urlPatternBase="/ws">
  <endpoint
    name="HelloWorld"
    displayName="Hello Service"
```

```
    description="A simple web service"
    interface="samples.webservices.jaxrpc.dynamic.HelloIF"
   implementation="samples.webservices.jaxrpc.dynamic.HelloImpl"/
  <endpointMapping

  endpointName="HelloWorld"
  urlPattern="/dynamic"/>
</webServices>>
```

The `<webServices>` element must contain one or more `<endpoint>` elements. In this example, note that the interface and implementation attributes of `<endpoint>` specify the service's interface and implementation class. The `<endpointMapping>` element associates the service port with an element of the endpoint URL path that follows the `urlPatternBase`.

For information about the XML schema for creating the runtime descriptor, see Appendix A, "XML Schema Definitions."

Namespace Mappings

This section is for developers who are familiar with WSDL, SOAP, and the JAX-RPC specifications.

Here is a schema type name example:

```
schemaType="ns1:SampleType"
xmlns:ns1="http://echoservice.org/types"
```

When generating a Java type from a schema type, `wscompile` gets the class name from the local part of the schema type name.

To specify the package name of the generated Java classes, you define a mapping between the schema type namespace and the package name. You define this mapping by adding a `<namespaceMappingRegistry>` element to the `config.xml` file. For example:

```
<service>

...

<namespaceMappingRegistry>

<namespaceMapping
namespace="http://echoservice.org/types" packageName="echoservice.org.types"/>
```

```
</namespaceMappingRegistry>

...

</service>
```

SOAP Handlers

A handler accesses a SOAP message that represents an RPC request or response. Handler class must implement the `javax.xml.rpc.handler` interface. A handler can manipulate a SOAP message with the APIs of the `javax.xml.soap` package.

The following are the examples of the tasks performed by a handler:

- Encryption and decryption
- Logging and auditing
- Caching
- Application-specific SOAP header processing

A handler chain is a list of handlers. You may specify one handler chain for the client and one for the server. On the client, you include the `<handlerChains>` element in the `jaxrpc-ri.xml` file. On the server, you include this element in the `config.xml` file.

Here is an example of the `<handlerChains>` element in the `config.xml` file:

```
<handlerChains>

<chain runAt="server"

roles= "http://acme.org/auditing
        http://acme.org/morphing"
   xmlns:ns1="http://foo/foo-1">

  <handler className="acme.MyHandler"

    headers ="ns1:foo ns1:bar"/>
    <property name="property" value="xyz"/>

  </handler>
```

```
  </chain>
</handlerChains>
```

Java Language Types Supported By JAX-RPC

JAX-RPC maps types of the Java programming language to XML/WSDL definitions. For example, JAX-RPC maps the `java.lang.String` class to the `xsd:string` XML data type. You as application developers need not know the details of these mappings, but you must be aware that not every class in the Java 2 Standard Edition (J2SE) can be used as a method parameter or return type in JAX-RPC.

J2SE SDK Classes

JAX-RPC supports the following J2SE SDK classes:

- `java.lang.Boolean`
- `java.lang.Byte`
- `java.lang.Double`
- `java.lang.Float`
- `java.lang.Integer`
- `java.lang.Long`
- `java.lang.Short`
- `java.lang.String`
- `java.math.BigDecimal`
- `java.math.BigInteger`
- `java.util.Calendar`
- `java.util.Date`

This release of JAX-RPC also supports several implementation classes of the `java.util.Collection` interface.The following table displays the supported classes. The first column lists the `java.util.Collection` subinterfaces, the second column lists the classes supported by the subinterface.

Table 2-3 Supported Classes

java.util.collection Subinterfaces	Classes Supported
`List`	Array List LinkedList Stack Vector
`Map`	HashMap Hashtable Properties TreeMap
`Set`	HashSet TreeSet

Primitives

JAX-RPC supports the following primitive types of the Java programming language:

- boolean
- byte
- double
- float
- int
- long
- short

Arrays

JAX-RPC also supports arrays with members of supported JAX-RPC types. Examples of supported arrays are int [] and String []. Also supports multidimensional arrays, such as BigDecimal [][].

Application Classes

JAX-RPC also supports classes that you have written for your applications. In an order processing application, for example, you might provide classes named Order, LineItem, and Product. The JAX-RPC Specification refers to such classes as value types, because their values (or states) may be passed between clients and remote services as method parameters or return values.

To be supported by JAX-RPC, an application class must conform to the following rules:

- It must have a public default constructor.
- It must not implement (either directly or indirectly) the `java.rmi.Remote` interface.
- Its fields must be supported JAX-RPC types.

The class may contain public, private, or protected fields. For its value to be passed (or returned) during a remote call, a field must meet these requirements:

- A public field cannot be final or transient.
- A non-public field must have corresponding getter and setter methods.

JavaBeans Components

JAX-RPC also supports JavaBeans components, which must conform to the same set of rules as application classes. In addition, a JavaBeans component must have a getter and setter method for each bean property. The type of the bean property must be a supported JAX-RPC type.

Copyright 2004 Sun Microsystems, Inc. All rights reserved.